

Laborator 2

Interpretorul de comenzi

2.1 Scopul lucrării

Lucrarea curentă abordează interpretorul de comenzi din Linux, împreună cu cele mai uzuale comenzi. Se discută de asemenea și fișierele de comenzi (eng. *scripts*).

2.2 Fundamente teoretice

O comandă dată interpretorului are un nume și poate avea zero, unul sau mai multe argumente.

Format general comenzi

<code>command_name arguments</code>

În mod implicit, terminalul curent execută comanda introdusă, citind eventualele date de la tastatură și afișând eventualele rezultate pe ecran. Pe durata execuției comenzii, terminalul este blocat (nu se mai pot tasta comenzi până ce comanda curentă nu se termină).

Fiecare comandă executată returnează un cod de ieșire (eng. *exit code*). Convenția este ca o comandă care s-a executat cu succes să returneze codul 0, iar în caz contrar să returneze o valoare nenulă.

Dacă se dorește execuția comenzii fără blocarea terminalului (în *background*), la finalul liniei se scrie caracterul '&'.
Exemplu: Comanda de mai jos lansează editorul **gedit** pentru a edita fișierul **test.c** în background (terminalul poate accepta noi comenzi în timpul editării).

Exemplu &

<code>gedit test.c &</code>

Se pot scrie mai multe comenzi pe aceeași linie, separate prin caracterul ';'. În acest caz, comenzile se vor executa succesiv, în ordinea în care au fost date. Mai multe comenzi se pot separa și prin:

- '|' (operatorul *pipe*): output-ul primei comenzi este folosit ca input pentru cea de-a doua;
- '&&' (operatorul *AND*): a doua comandă se execută doar dacă prima comandă s-a executat cu succes (*exit code*-ul este 0, considerat TRUE);
- '||' (operatorul *OR*): a doua comandă se execută doar dacă prima comandă nu s-a executat cu succes (*exit code*-ul este diferit de 0, considerat FALSE).

Exemple: În prima linie din exemplele de mai jos, comanda **grep** extrage din fișierul **test.c** doar liniile care conțin subșirul "example". Aceste linii extrase ajung ca input la comanda **wc -l** (**wc**=*word count*, **-l**=*lines*), care numără câte astfel de linii avem. Pe ecran se va vedea doar numărul de linii. În a doua linie fișierul **test.c** se compilează folosind programul **gcc**, iar executabilul obținut se rulează doar dacă compilarea s-a efectuat cu succes.

Exemple comenzi multiple

```
grep example test.c | wc -l
gcc test.c -o test && ./test
```

Input-ul sau output-ul unei comenzi pot fi redirectate cu ajutorul operatorilor:

- ‘<’ (redirectare *standard input*, adică ceea ce ar fi introdus de la tastatură);
- ‘>’ (redirectare *standard output*, adică ceea ce ar fi afișat pe ecran, ca mesaj normal);
- ‘2>’ (redirectare *standard error*, adică ceea ce ar fi afișat pe ecran, ca mesaj de eroare).

Exemplu: Prima comandă din cele de mai jos, citește linie cu linie din fișierul “file.txt” (folosit ca redirectare a intrării standard) și afișează liniile citite pe ecran. Al doilea exemplu folosește **ls** pentru a lista conținutul folderului **/etc**. În loc să se afișeze pe ecran, conținutul respectiv se scrie în fișierul **contents.txt**. Mesajele de eroare vor fi ignorate prin redirectarea acestora în fișierul **/dev/null** (ce poate fi privit ca o “gaură neagră” a sistemului de fișiere).

Exemplu redirectare

```
while read line; do echo $line; done < file.txt$
ls /etc > contents.txt 2> /dev/null
```

2.2.1 Comenzi uzuale

În laboratorul anterior s-au discutat câteva comenzi de bază, pentru familiarizarea cu terminalul. Pe lângă acestea, mai avem următoarele comenzi care deasemenea se utilizează în mod frecvent. Sintaxa completă a acestora se găsește în paginile de manual ale sistemului (rulați **man nume_comandă**).

cp (*copy*) copiază unul sau mai multe fișiere dintr-o locație în alta.

Exemplu cp

```
cp /etc/passwd my_passwd_copy.txt
```

Comanda de mai sus copiază fișierul **/etc/passwd** (cale absolută, deoarece începe cu ‘/’) în folderul curent, cu numele **my_passwd_copy.txt**. Primul parametru al comenzii este fișierul sursă, iar al doilea este fișierul destinație.

mv (*move*) mută unul sau mai multe fișiere dintr-o locație în alta.

Exemplu mv

```
mv my_passwd_copy.txt users_list.txt
```

În exemplul de mai sus, fișierul **my_passwd_copy.txt** este mutat în **users_list.txt**. Operația de mutare poate fi privită ca o operație de redenumire. La fel ca la comanda **cp**, primul parametru este sursa, iar al doilea este destinația.

less ne permite să vizualizăm în terminal un fișier text.

Exemplu less

```
less users_list.txt
```

less permite vizualizarea într-un mod interactiv, folosind tastele săgeți, “Page Up”, “Page Down”, etc. Pentru a încheia vizualizarea se apasă tasta “q”.

grep caută un anumit conținut într-un fișier.

Exemple grep

```
grep root users_list.txt
grep -v root users_list.txt
```

În primul exemplu se afișează liniile care conțin string-ul “root” din fișierul **users_list.txt**. În al doilea exemplu se afișează liniile care nu conțin string-ul căutat (parametrul **-v** înseamnă inversarea căutării).

wc (*word count*) numără cuvinte, linii sau caractere într-un fișier.

Exemple wc

```
wc users_list.txt
wc -l users_list.txt
wc -w users_list.txt
wc -m users_list.txt
```

Primul exemplu afișează atât numărul de linii, cât și numărul de cuvinte, respectiv caractere din fișierul `users_list.txt` primit ca parametru. Următoarele exemple folosesc opțiunile `-l`, `-w` și `-m` pentru a afișa aceste informații pe rând.

cut ajută la filtrarea informației dintr-un output.

Exemple cut

```
cut -d: -f 1,6 /etc/passwd
ls -l | cut -c 2,3,4
```

În primul exemplu, folosim comanda `cut` ca să extragem câmpurile 1 și 6 (opțiunea *fields* `-f 1,6`) de pe fiecare linie a fișierului `/etc/passwd`, considerând că separatorul de câmpuri este `:` (opțiunea *delimiter* `-d:`).

În al doilea exemplu, output-ul comenzii `ls -l` este filtrat de `cut`, extrăgând al 2-lea, al 3-lea și al 4-lea caracter de pe fiecare linie (opțiunea *characters* `-c 2,3,4`). Aceste caractere corespund permisiunilor proprietarului pentru fiecare fișier.

find caută fișiere într-o ierarhie de directoare.

Exemple find

```
find . -name "*.txt"
find /usr/bin -type f -perm 0755
```

Primul argument reprezintă directorul în care să se facă căutarea. În primul exemplu, se face căutarea în directorul curent (`.`), iar în al doilea în directorul `/usr/bin`. În primul exemplu, căutăm toate elementele care se termină cu `.txt`. În al doilea exemplu, căutăm doar fișiere (`-type f`) care au permisiunile 0755.

2.2.2 Variabile

În interacțiunea cu terminalul, putem utiliza variabile pentru a reține anumite valori (de tip string) și pentru a le folosi în alte comenzi. Pentru a inițializa o variabilă, scriem numele acesteia, urmat de simbolul `=`, urmat de valoare pe care vrem să o atribuim (fără spații înainte sau după `=`). Pentru a folosi variabila respectivă, numele acesteia trebuie prefixat cu simbolul `$`. În exemplul de mai jos, variabila `NAME1` se inițializează cu string-ul `"Tom"`, apoi se folosește în comanda `echo`, împreună cu variabila `NAME2` (se va afișa `Hello Tom and Bill`).

Exemplu inițializare și utilizare variabilă

```
NAME1=Tom
export NAME2=Bill
echo "Hello $NAME1 and $NAME2"
```

Variabilele `NAME1` și `NAME2` sunt asociate doar terminalului curent. Într-un alt terminal, acestea nu mai sunt vizibile. Dacă din terminalul curent este pornit un proces nou, acesta moștenește `NAME2` (deoarece a fost exportată), dar nu și `NAME1`.

2.2.2.1 Variabile de mediu

Pe lângă variabilele definite de utilizator, terminalul mai utilizează și variabile de mediu (eng. *environment variables*), ce pot fi privite ca niște variabile globale, la nivelul terminalului.

Exemple de variabile de mediu:

- **USER** - conține numele utilizatorului curent
- **HOME** - conține calea spre folderul personal al utilizatorului curent
- **PATH** - conține o listă de căi în sistemul de fișiere, separate prin ":". Atunci când se rulează o comandă în terminal, fără a i se specifica calea absolută, executabilul acesteia se caută în fiecare folder din **PATH** pe rând, utilizându-se primul în care executabilul se găsește.

Pentru a vizualiza toate variabilele de mediu pentru terminalul curent, se poate rula comanda `printenv`.

2.2.3 Fișiere de comenzi

Fișierele de comenzi, numite și *script*-uri conțin o secvență de comenzi, care se execută pe rând. Principala utilizare a acestora este automatizarea unor sarcini. Pe lângă execuția de comenzi, un script mai poate conține și structuri de control (`if`, `while`, `for`) și chiar proceduri.

2.2.3.1 Crearea și execuția

Pentru a crea un script, pur și simplu creăm un fișier (prin convenție, script-urile shell au extensia `.sh`, dar acest lucru nu este obligatoriu) și scriem în el comenzi. Script-urile nu se compilează, ci se execută direct, fiind interpretate de către shell. Pentru editare, putem folosi un editor în mod text (ex. `nano`, `vim`) sau unul vizual (ex. `gedit`, `sublime`). În exemplul de mai jos se creează fișierul `myscript.sh` folosind comanda `touch`, care apoi se deschide în editorul `gedit` în mod asincron (pentru a nu ține blocat terminalul).

Crearea și editarea unui script

```
touch myscript.sh
gedit myscript.sh &
```

Ca să executăm un script, fie îl dăm ca parametru interpretorului de comenzi (`bash`), fie folosim `chmod` ca să îl facem executabil, apoi îl rulăm direct.

Execuția unui script - varianta 1

```
bash myscript.sh
```

Execuția unui script - varianta 2

```
chmod +x myscript.sh
./myscript.sh
```

Comanda `chmod` trebuie rulată o singură dată pentru acordarea dreptului de execuție, chiar dacă script-ul se modifică. La rularea directă, ar trebui să specificăm și ce interpretor trebuie folosit. Pentru asta, prima linie a script-ului poate avea o construcție specială, numită *shebang*. Linia va începe cu `"#!"` (simbolul `"#"` se folosește în mod normal pentru comentarii, în script-uri) și se va continua cu calea către interpretor, precum în exemplul de mai jos (linia 1).

```
1 |#!/bin/bash
2 |
3 |grep nologin /etc/passwd > file1.txt
4 |COUNT1=`wc -l file1.txt | cut -d" " -f 1`
5 |echo "We have $COUNT1 users that cannot login."
```

În linia 3 a script-ului se rulează comanda `grep`, care caută textul `"nologin"` în fișierul `/etc/passwd` și salvează liniile care îl conțin în fișierul `file1.txt`. Pe linia 4 se rulează o comandă care are ca output numărul de linii a fișierului, iar rezultatul se salvează în variabila `COUNT1`. Pentru ca variabila să rețină rezultatul comenzii, nu textul acesteia, s-a pus comandă între apostroafe inverse (`" "` - tasta de sub *Esc*). La linia 5 se afișează un text ce conține variabila `COUNT1`.

2.2.3.2 Utilizarea argumentelor

Un script poate primi argumente în linia de comandă, pe care le poate accesa prin parametrii poziționali `$1`, `$2`, ..., `${10}`, ..., `$#` indică numărul de argumente primite, iar `$@` va conține toată lista de parametri.

În exemplul de mai jos se vede un exemplu de folosire a argumentelor primite în linia de comandă.

```
| echo "We have $# arguments. The first one is $1."
```

2.2.3.3 Structuri de control

În fișiere de comenzi se pot folosi structuri de control, ca și în alte limbaje de programare. În continuare, se va prezenta sintaxa pentru `if` și `for`.

```
| if cond1
| then
|     cmd_list1
| elif cond2
| then
|     cmd_list2
| else
|     cmd_list3
| fi
```

La `if`, clauzele `elif` și `else` sunt opționale. În exemplul de mai jos, se verifică dacă primul argument primit în linia de comandă este mai mare decât 3.

```
1 | if test $1 -gt 3
2 | then
3 |     echo "Greater"
4 | else
5 |     echo "Smaller or equal"
6 | fi
```

Pentru verificarea condiției am folosit comanda `test`. Nu putem folosi operatorul `>` pentru comparație, deoarece acesta se folosește deja la redirectarea output-ului. În schimb, vom folosi `-gt` (eng. *greater than*).

Sintaxa pentru `for` este ilustrată mai jos.

```
| for name in list
| do
|     command_list
| done
```

În exemplul de mai jos se parcurg toate fișierele cu extensia `".txt"` din directorul curent (linia 1), li se calculează dimensiunea în bytes (`-b`) folosind comanda `du` (linia 3), iar dacă aceasta este mai mare decât argumentul primit (linia 4), se afișează numele fișierului (linia 6).

```
1 | for FNAME in *.txt
2 | do
3 |     FSIZE=`du -b $FNAME | cut -f 1`
4 |     if test $FSIZE -gt $1
5 |     then
6 |         echo $FNAME
7 |     fi
8 | done
```

2.3 Mersul lucrării

2.3.1 Probleme rezolvate

1. Dacă rulăm al doilea exemplu pentru comanda **cut** din secțiunea 2.2.1 observăm că prima linie nu corespunde formatului prezentat, deoarece comanda **ls -l** afișează pe prima linie un total. Modificați comanda, astfel încât să se afișeze rezultatul dorit.

Soluția 1: Putem insera o comandă **grep**, care să elimine liniile care încep cu "total". Comanda se va modifica în:

```
ls -l | grep -v "^total" | cut -c 2,3,4
```

Parametrul **-v** pentru **grep** afișează liniile care nu conțin textul căutat, iar caracterul **"^"** indică potrivirea doar la începutul unei linii (nu dorim să eliminăm, de exemplu, fișierele al căror nume conține șirul de caractere "total").

Soluția 2: Folosim comanda **tail** cu parametrul **-n +2**, care afișează toate liniile primite, începând de la a 2-a:

```
ls -l | tail -n +2 | cut -c 2,3,4
```

2. Scrieți un script care afișează argumentele primite în linia de comandă, în ordine inversă.

Soluție: Pentru a parcurge argumentele primite în lina de comandă, se folosește următoarea buclă:

```
1 | for ARG
2 | do
3 |     echo $ARG
4 | done
```

Problema este că în codul de mai sus, argumentele se parcurg în ordine, iar noi avem nevoie de ele în ordine inversă. Prin urmare, vom construi un string și vom insera fiecare argument, pe rând, în fața șirului, pentru a obține ordinea inversă. La final se afișează string-ul construit.

```
1 | RESULT=""
2 | for ARG
3 | do
4 |     RESULT="$ARG $RESULT"
5 | done
6 | echo $RESULT
```

2.3.2 Probleme propuse

1. (15p) Afișați pe ecran doar linia din **/etc/passwd** care corespunde utilizatorului curent.
2. (15p) Afișați liniile dintr-un fișier care nu conțin nici string-ul "text1", nici string-ul "text2".
3. (20p) Scrieți o comandă care numără câte elemente se găsesc într-un director. Câte dintre acestea sunt la rândul lor directoare?
4. (25p) Scrieți un script care afișează toate fișierele cu extensia **.c** din folderul curent, care au peste 20 de linii. Modificați script-ul astfel încât să fie luate în considerare și fișierele din subdirectoare (recursiv). *Indiciu:* comanda **find** parcurge folderele recursiv.
5. (25p) Scrieți un script care primește ca argument în linia de comandă unul din cuvintele "read", "write" sau "execute". Pentru fiecare fișier din directorul curent care are permisiunea respectivă se va crea un director cu același nume, la care se adaugă extensia **.dir**. În directorul respectiv se va scrie un fișier numite "size.txt" ce va conține dimensiunea fișierului.